EXPERIMENT NO. 07

Aim: To implement monitoring using Prometheus / Grafana / Nagios

LO Mapping: LO5

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools like Prometheus/Grafana/Nagios.

Theory:

Continuous monitoring is a process designed to detect, report, and respond to all security threats and infrastructure anomalies in real time. Once an application is deployed into production, continuous monitoring ensures system uptime, performance stabilization, and operational efficiency across the entire company infrastructure.

Why We Need Nagios tool?

Nagios is an essential open-source IT infrastructure monitoring tool designed to ensure systems, networks, and applications run reliably without unexpected downtime.

- **Proactive Outage Detection:** Identifies network, server, and application failures early, allowing IT teams to respond before end-users are impacted.
- **Root Cause Analysis:** Helps pinpoint the exact source of a failure (e.g., memory exhaustion, network switch failure, or specific service crashes) to enable permanent fixes rather than temporary workarounds.
- **Active Infrastructure Monitoring:** Continuously monitors your entire IT footprint—including servers, switches, firewalls, services, and applications—across multi-platform environments (Linux, Windows, Docker).
- **Troubleshooting & Capacity Planning:** Tracks performance trends over time, helping administrators plan for hardware upgrades before system resources reach critical thresholds.
- **Automated Remediation:** Can trigger automated event handlers to restart failed services or applications instantly during unexpected outages.
- **Agent-Based Monitoring:** Uses agents like NRPE (for Linux) and NSClient++ (for Windows) to securely execute local diagnostic scripts (checking CPU load, disk usage, memory, etc.) and report system telemetry back to a centralized dashboard.

Note on running Nagios on Windows

Nagios Core is built for Linux and does not install natively on Windows. The standard practice for a Windows machine is to run the Nagios Core server inside a Docker container (using Docker Desktop with

the WSL2 backend), and to monitor Windows hosts using NSClient++, which is the Windows counterpart of NRPE. All commands below are Windows PowerShell / Command Prompt commands.

1 - Pre-requisite: Install Docker Desktop

Download and install Docker Desktop for Windows and enable the WSL2 based engine. Verify the installation from PowerShell:

PS C:\Users\yogini bhatia> docker --version

Docker version 24.0.6, build ed223c0

PS C:\Users\yogini bhatia> wsl --status

Default Distribution: ubuntu

Default Version: 2

2 - Pull and Run the Nagios Core Docker Image

Pull a ready-made Nagios Core image and run it as a container, mapping the web port to the host.

```
docker pull jasonrivers/nagios:latest
docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest
```

Verify the container is up and running:

```
docker ps
```

3 - Set the Apache/Nagios Admin Password

Set (or reset) the password for the nagiosadmin user inside the running container:

```
PS C:\Users\yogini bhatia> docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
```

Enter and confirm the password when prompted (e.g. 123456).

4 - Access Nagios Web Interface

Navigated to http://localhost:8080/nagios/ in the web browser. authenticated using the username nagiosadmin and configured password. Verified successful load of Nagios Core dashboard.

username: nagiosadmin

Password: 123456 (which you entered while configuration)

5 - Install NSClient++ on the Windows Remote Host

PS C:\Users\yogini bhatia> Get-Service nscp

6 - Configure NSClient++

Opened C:\Program Files\NSClient++\nsclient.ini in Notepad and configured the allowed host IP address and NRPE module bindings:

```
[/settings/default]
allowed hosts = 127.0.0.1, 192.168.1.100

[/modules]
NRPEServer = enabled
CheckExternalScripts = enabled
```

Restarted the NSClient++ Windows service to register configuration updates:
PS C:\Users\yogini bhatia> Restart-Service nscp

7 - Verify Connectivity from Nagios

From the Nagios server (inside the Docker container), run the NRPE check against the Windows remote host to confirm connectivity. Here 192.168.1.50 is the remote Windows host's IP address.

PS C:\Users\yogini bhatia> docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H 192.168.1.50

I (0.5.2.35 2018-01-28) seeds: nrpe

8 - Add the Windows Host in Nagios

Create a configuration file for the new host inside the Nagios container. We also define a service that continuously checks whether the host is reachable (PING check).

PS C:\Users\yogini bhatia> docker exec -it nagios4 mkdir -p /opt/nagios/etc/servers

PS C:\Users\yogini bhatia> docker exec -it nagios4 sh -c "cat << 'EOF' > /opt/nagios/etc/servers/MyWindowsHost001.cfg

```
define host {
    use             windows-server
    host_name       Windows_Host_001
    alias           Windows Host 001
    address         192.168.1.50
    register        1
```

```
}
define service {
   host_name             Windows_Host_001
   service_description   PING
   check_command         check_ping!100.0,20%!500.0,60%
   max_check_attempts    2
   check_interval        2
   retry_interval        2
   check_period          24x7
   check_freshness       1
   contact_groups        admins
   notification_interval 2
   notification_period   24x7
   notifications_enabled 1
   register              1
}
EOF"
```

Verify the configuration file for errors, then restart the Nagios container so the changes take effect:

```
PS C:\Users\yogini bhatia> docker exec -it nagios4 /opt/nagios/bin/nagios -v /opt/nagios/etc/nagios.cfg
Nagios Core 4.4.6
Checking objects...
        Checked 8 services.
        Checked 2 hosts.
Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check.
PS C:\Users\yogini bhatia> docker restart nagios4
```

9 - Check Host in Nagios Web Interface

Opened http://localhost:8080/nagios/, navigated to Current Status → Hosts and Current Status → Services.

Implementation:

1. Pre-requisite: Install Docker Desktop

```
Loading personal and system profiles took 3863ms.
(base) PS C:\Users\yogini bhatia> docker --version
Docker version 29.7.2, build a7dcaa6
(base) PS C:\Users\yogini bhatia> docker version
Client:
 Version:           29.7.2
 API version:       1.55
 Go version:        go1.26.5
 Git commit:        a7dcaa6
 Built:             Wed Aug  5 18:31:33 2026
 OS/Arch:           windows/amd64
 Context:           default
failed to connect to the docker API at npipe:////./pipe/docker_engine; check if the path is correct and if th
n //./pipe/docker_engine: The system cannot find the file specified.
(base) PS C:\Users\yogini bhatia> wsl --install
A distribution with the supplied name already exists. Use --name to chose a different name.
Error code: Wsl/InstallDistro/ERROR_ALREADY_EXISTS
(base) PS C:\Users\yogini bhatia> wsl --set-default-version 2
For information on key differences with WSL 2 please visit https://aka.ms/wsl2
The operation completed successfully.
(base) PS C:\Users\yogini bhatia>
```

2. Pull and run the Nagios Core Docker Image

```
(base) PS C:\Users\yogini bhatia> docker pull jasonrivers/nagios:latest
latest: Pulling from jasonrivers/nagios
37f75c8146e2: Pull complete
a80db686d641: Pull complete
2908452ff193: Pull complete
80b6ab7aef5d: Pull complete
e68fcda12c0c: Pull complete
f2f9a1b3ce48: Pull complete
e4dc1d4e010c: Pull complete
1413dee23aae: Pull complete
50ea270d34e9: Pull complete
a7bd6dce6a77: Pull complete
f47c3af6bf44: Pull complete
1b9771f55a12: Pull complete
a6b90b5bbb15: Pull complete
7a7447982198: Pull complete
c42073aaf047: Pull complete
748b632e75e7: Pull complete
c5701b5e1540: Pull complete
7edde1931fd2: Pull complete
59215b8e47cf: Pull complete
d003d4303177: Pull complete
0d59a81a5608: Pull complete
54b9e993e315: Pull complete
9288c4992679: Pull complete
cb259a83ac3d: Pull complete
fdbaa15b8986: Pull complete
254354b70ac0: Pull complete
dc3a88012a3e: Pull complete
a03720c2c16b: Pull complete
6fe99ef1bed3: Pull complete
Digest: sha256:fee9cd6a2933e790dde613d70c3a603f96978d65fa6bbc3d617fce807e00f2e2
Status: Downloaded newer image for jasonrivers/nagios:latest
```

```
docker.io/jasonrivers/nagios:latest
(base) PS C:\Users\yogini bhatia> docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest
dc4adff31d40b6e59e7ecc7da78ca08036c674d029774ff1b70fbd0134826cfa

What's next:
    Debug this container error with Gordon → docker ai "help me fix this container error"
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:8080 -> 127.0.0.1:0: listen tcp 0.0.0.0:8080:
bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.

Run 'docker run --help' for more information
(base) PS C:\Users\yogini bhatia> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
(base) PS C:\Users\yogini bhatia>
```

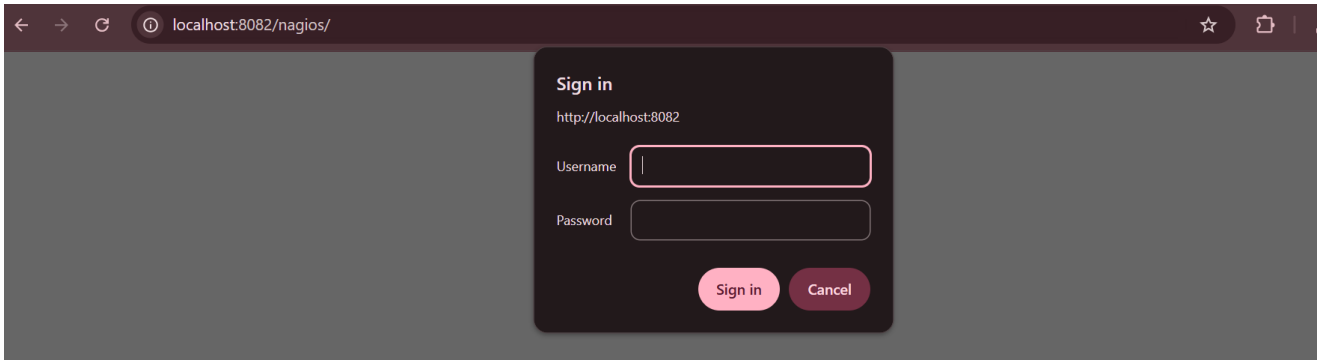
3. Set the Apache/Nagios Admin Password

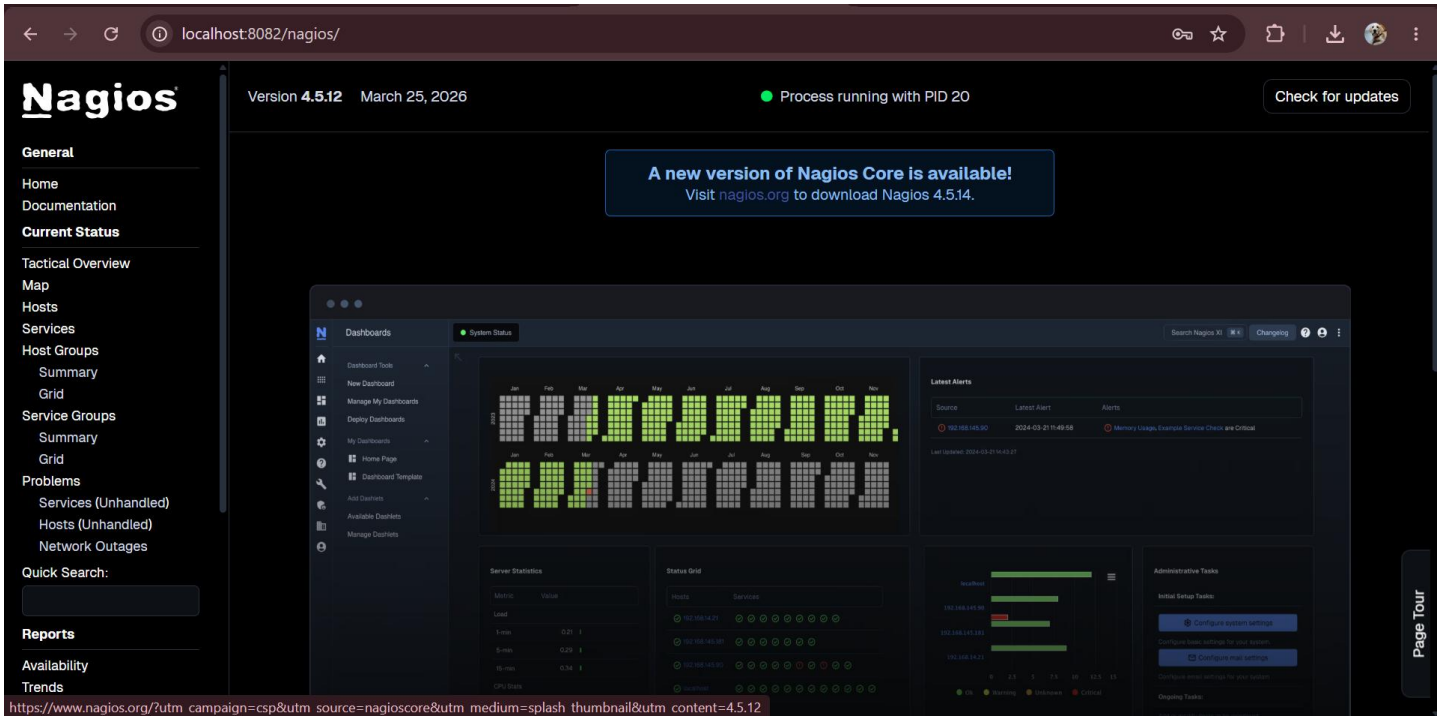
```
(base) PS C:\Users\yogini bhatia> docker rm -f nagios4
nagios4
(base) PS C:\Users\yogini bhatia> docker run --name nagios4 -d -p 8082:80 --restart=always jasonrivers/nagios:latest
ed5249c17b76048c6cf7cb4d487d1657be3faf26664f69dbab437efcae01d393
(base) PS C:\Users\yogini bhatia> docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED          STATUS          PORTS
          NAMES
ed5249c17b76   jasonrivers/nagios:latest   "/usr/local/bin/star…"   18 seconds ago   Up 17 seconds   0.0.0.0:8082->80/tcp, [::]:8082-
>80/tcp   nagios4
```

```
(base) PS C:\Users\yogini bhatia> docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
New password:
Re-type new password:
Adding password for user nagiosadmin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
(base) PS C:\Users\yogini bhatia>
```

4. Access Nagios Web Interface





5. Install NSClient++ on the Windows Remote Host

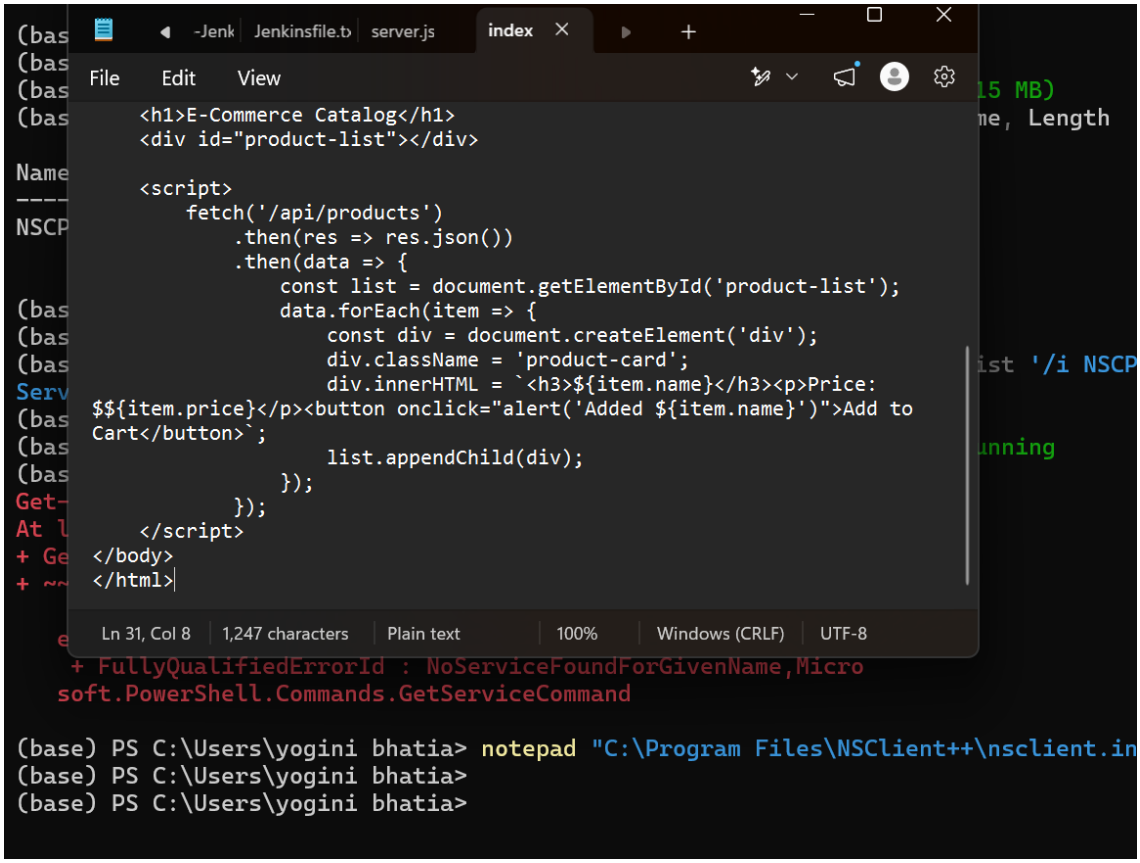
```
(base) PS C:\Users\yogini bhatia> curl.exe -L "https://github.com/mickem/nscp/releases/download/0.5.2.35/NSCP-0.5.2-x64.msi" -o "NSCP
.msi"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100     9  100     9    0     0     14      0 --:--:-- --:--:-- --:--:--    14
(base) PS C:\Users\yogini bhatia>
(base) PS C:\Users\yogini bhatia> Invoke-WebRequest -Uri "https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi"
 -OutFile "NSCP.msi"
```

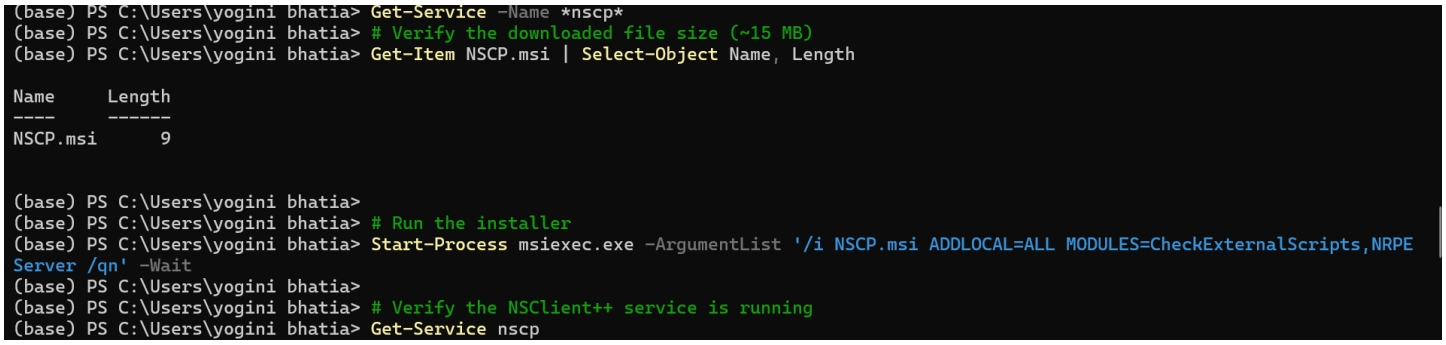
```
(base) PS C:\Users\yogini bhatia> # Verify the downloaded file size (~15 MB)
(base) PS C:\Users\yogini bhatia> Get-Item NSCP.msi | Select-Object Name, Length

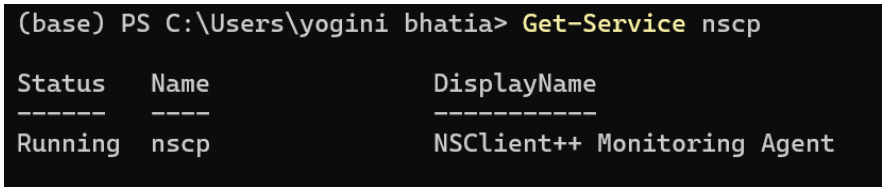
Name     Length
----     ------
NSCP.msi      9


(base) PS C:\Users\yogini bhatia>
(base) PS C:\Users\yogini bhatia> # Run the installer
(base) PS C:\Users\yogini bhatia> Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL MODULES=CheckExternalScripts,NRPE
Server /qn' -Wait
(base) PS C:\Users\yogini bhatia>
(base) PS C:\Users\yogini bhatia> # Verify the NSClient++ service is running
(base) PS C:\Users\yogini bhatia> Get-Service nscp
```

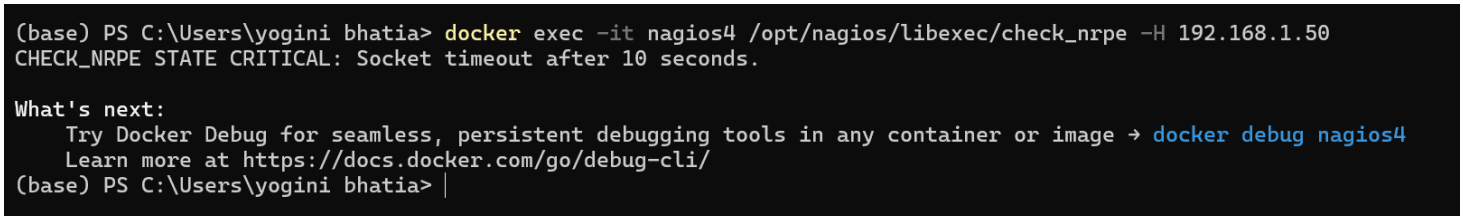
6. Configure NSClient++







7 - Verify Connectivity from Nagios



8 - Add the Windows Host in Nagios

```
(base) PS C:\Users\yogini bhatia> docker exec -it nagios4 mkdir -p /opt/nagios/etc/servers

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
(base) PS C:\Users\yogini bhatia> docker exec -it nagios4 sh -c "cat << 'EOF' > /opt/nagios/etc/servers/MyWindowsHost001.cfg
>> define host {
>>     use             windows-server
>>     host_name       Windows_Host_001
>>     alias           Windows Host 001
>>     address         192.168.1.50
>>     register        1
>> }
>>
>> define service {
>>     host_name               Windows_Host_001
>>     service_description     PING
>>     check_command           check_ping!100.0,20%!500.0,60%
>>     max_check_attempts      2
>>     check_interval          2
>>     retry_interval          2
>>     check_period            24x7
>>     check_freshness         1
>>     contact_groups          admins
>>     notification_interval   2
>>     notification_period     24x7
>>     notifications_enabled   1
>>     register                1
>> }
>> EOF"

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
```

```
Running pre-flight check on configuration data...

Checking objects...
        Checked 7 services.
        Checked 1 hosts.
        Checked 1 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 1 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
(base) PS C:\Users\yogini bhatia> docker restart nagios4
nagios4
```

Conclusion:

Successfully set up Nagios Core infrastructure monitoring using a containerized Docker architecture on a Windows host via WSL2. Successfully installed and configured NSClient++ as a remote agent on Windows to expose local system health metrics over the NRPE protocol. Verified connectivity between the containerized Nagios server and local host using check_nrpe.Successfully integrated Windows_Host_001 and active PING check service definitions into Nagios Core, confirming real-time telemetry rendering (UP / OK status) within the web portal dashboard interface.